1) Jesus Alonso Chalco Luque
2) Celia Marina Mamani Ccora

1. Ranking con función de ventana vs. subconsulta correlacionada.

Consigna (código en php MyAdmin):

Escribe en phpMyAdmin una consulta con RANK() OVER(ORDER BY...) que le asigne a cada jugador su posición en el leaderboard global. por mejor puntaje, manejando empates. Luego escribe una segunda consulta que logre el mismo resultado usando subconsulta correlacionada (sin RANK).

```
SELECT
    id_jugador,
    nombre,
    puntaje,
    RANK() OVER (ORDER BY puntaje DESC) AS posición
FROM
    jugadores;
```

2. Jugadores "fantasma" (anti-join)

Consigna (código en phpMyAdmin):
Escribe una consulta que encuentre todos los jugadores que nunca han obtenido ningun item (no aparecen en Inventario_Items) usando LEFT JOIN ... IS NULL, y otra equivalente con NOT EXISTS.

1. Con LEFT JOIN ... IS NULL

```
SELECT j.*
FROM Jugadores j
LEFT JOIN Inventario_items i ON j.jugador_id = i.jugador_id
WHERE i.jugador_id IS NULL;
```

2 Con NOT EXIST (equivalente)

```
SELECT j.*
    FROM Jugadores j
    WHERE NOT EXIST(
        SELECT 1
    FROM Inventario_Items i
    WHERE i.jugador_id = j.jugador_id
    );
```

Pregunta 8. Roles y Privilegios.
Pregunta 3. Vista restringida y principio de menor privilegio.
Consigna (código en phpMyAdmin):

El usuario analista_da necesita un reporte de actividad de jugadores, pero no debe poder ver la columna email (dato sensible). Crea una VIEW que excluya email, otorga SELECT sobre la vista analista_da, y revoca el SELECT directo que tenía sobre la tabla jugadores

(3)

```
CREATE OR REPLACE VIEW vista_actividad_jugadores AS
SELECT
    jugador_id,
    nombre,
    monedas
FROM jugadores;

GRANT SELECT ON vista_actividad_jugadores TO 'analista_da'@'%';
REVOKE SELECT ON jugadores FROM 'analista_da'@'%';
```

(4) Provocar un deadlock real (no solo evitarlo)

Consigna (código en phpMyAdmin):

Abre dos pestañas de phpMyAdmin. En la
Pestaña A: START TRASACTION; UPDATE
Jugadores SET monedas = monedas - 10 WHERE jugador_id = 1;
(sin COMMIT).
En la pestaña B: START TRANSACTION; UPDATE
Jugadores SET monedas = monedas - 10 WHERE jugador_id = 2; y
luego intenta actualizar también el jugador 1. En la pestaña A,
intenta actualizar también el jugador 2. Provoca el error
1213 (deadlock).

- Pestaña A:

```
START TRANSACTION;
UPDATE jugadores SET monedas = monedas - 10 WHERE jugador_id = 2
```

(deja la pestaña así, sin hacer COMMIT)

- Pestaña B:

```
START TRANSACTION;
UPDATE jugadores SET monedas = monedas - 10 WHERE jugador_id = 2;
```

(deja la pestaña así, sin hacer COMMIT)

- Pestaña B

```
UPDATE jugadores SET monedas = monedas - 10 WHERE jugador_id = 1;
```

(esta sentencia se quedará esperando... no la canceles)

- Pestaña A

```
UPDATE jugadores SET monedas = monedas - 10 WHERE jugador_id = 2;
```

En este momento Mysql detectara el dead lock y una de las 2 transacciones recibira error

- Error 1213 (40001): Deadlock found when trying to get lock; try restarting transaction

Pregunta 5. Condición de carrera sin FOR UPDATE
Consigna (código en php My Admin):

Toma el procedimiento comprar_item_unico y crea una copia modificada que quite la cláusula FOR UPDATE del SELECT. Ejecuta el mismo experimento de dos compras simultáneas (Script 3 de la Sesión 3) con esta versión insegura.

VERSION SEGURA:

```
SELECT disponible FROM items WHERE id = X FOR UPDATE;
```

VERSIÓN INSEGURA:

```
SELECT disponible FROM items WHERE id = X;
```

Pregunta 6: Inyección con sentencias apiladas (stacked queri

Usando PREPARE / EXCUTE para simular cómo un backend ma
hecho concatenando texto, construye el payload '; UPDATE
Jugadores SET monedas = 999999 WHERE nombre_usuario = 'NoobMa
-- contra una consulta dinámica concatenada, y luego el mismo payload
contra buscar_jugador_seguro.

1. Simulación contra una consulta dinámica concatenada (vulnerabl

```
SET @ payload = "' ; UPDATE jugadores SET monedas =
999999 WHERE nombre_usuario = 'Noob Master'; -- ";

SET @ query_vulnerable = CONCAT ("SELECT * FROM jugadores
WHERE nombre_usuario = '", @ payload ) ;
```

Pregunta 6: Inyección con sentencias apiladas (stacked queries)

Usando PREPARE / EXCUTE para simular cómo un backend mal hecho concatenaría texto, construye el payload '; UPDATE jugadores SET monedas = 999999 WHERE nombre_usuario = 'NoobMaster'; -- contra una consulta dinámica concatenada, y luego el mismo payload contra buscar_jugador_seguro.

1. Simulación contra una consulta dinámica concatenada (vulnerable)

```
SET @ payload = "' '; UPDATE jugadores SET monedas =
999999 WHERE nombre_usuario = 'NoobMaster'; -- ";

SET @query_vulnerable = CONCAT ("SELECT * FROM jugadores
WHERE nombre_usuario = '", @ payload );

PREPARE stmt_vulnerable FROM @query_vulnerable;
EXECUTE stmt_vulnerable;
DEALLOCATE PREPARE stmt_vulnerable;
```

2. Simulación contra buscar_jugador_seguro (Protegida)

```
SET @ payload = "' '; UPDATE jugadores SET monedas = 999999 WHERE
nombre_usuario = 'NoobMaster'; -- ";

SET @ query_segura = " SELECT * FROM jugadores WHERE
nombre_usuario = ? ";

PREPARE stmt_seguro FROM @ query_segura;
EXECUTE stmt_seguro USING @ payload;
DEALLOCATE PREPARE stmt_seguro;
```

Pregunta 7. Anti-trampas con CHECK vs. TRIGGER.

Escribe una restricción CHECK que impida que monedas sea negativo, y además un TRIGGER BEFORE UPDATE que rechace (SIGNAL SQLSTATE '45000') cualquier intento de subir el nivel de un jugador en más de 1 por operación.

(1) Defensa Oral (qué debes explicar frente al grupo): Explica la diferencia entre integridad declarativa (CHECK, evaluado por el motor) y pro lógica personalizada), y porque el anti-trampas de nivel necesitaba un trigger y no ba

1. Restricción CHECK (Monedas no negativas)

```
ALTER TABLE jugadores
ADD CONSTRAINT chk_monedas_no_negativas
CHECK (monedas >= 0);
```

2. TRIGGER BEFORE UPDATE (Límite de subida de nivel).

```
CREATE TRIGGER tg_anti_trampas_nivel
BEFORE UPDATE ON jugadores
FOR EACH ROW
BEGIN
    IF NEW.nivel > OLD.nivel + 1 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT='Error anti-trampas: No
        puedes subir más de 1 nivel por operación.';
    END IF;
END
```

Pregunta 8: Agregar una partición sin perder datos y probar partition pruning.

Sobre Partidas_Fragmentada, agrega una tercera partición p_asia VALUES IN ('ASIA') usando ALTER TABLE ... ADD PARTITION, inserta un registro de prueba con region = 'ASIA' y corre EXPLAIN filtrando por esa región.

1. Agregar la nueva partición sin perder datos

```
ALTER TABLE Partidas_Fragmentada
ADD PARTITION (
    PARTITION p_asia VALUES IN ('ASIA')
);
```

2. Insertar el registro de prueba

```
INSERT INTO Partidas_Fragmentada (region,
  fecha, estado)

VALUES ('ASIA', NOW(), 'Completada');
```

3. Probar el Partition Pruning con EXPLAIN

```
EXPLAIN SELECT * FROM Partidas_Fragmentada
  WHERE region = 'ASIA';
```

Pregunta 9. Simular ataque y recuperar con punto en el tiempo (PITR)

Haz un UPDATE destructivo simulando un ataque (por ejemplo, poner monedas = 0 a todos los Usando el backup completo y el binlog de la sesión 6, restaura la base al estado justo antes de perder transacciones legítimas posteriores a tu último backup pero anteriores al ataque.

1. Simular el ataque destructivo (SQL)

```
UPDATE jugadores SET monedas = 0;
```

2. Identificar el punto del ataque en el Binlog (Consola / Terminal)

```
bash
mysqlbinlog --base64-output=DECODE-ROWS -v /
var/lib/mysql/mysql-bin.000006 | grep -C5 -i
"UPDATE jugadores"
```

3. Restaurar el Backup Completo (consola).

```
mysql -u root -p juego < backup_completo.sql
```

4. Aplicar PITR hasta antes del ataque

Opción A: Por posición

```
mysqlbinlog --stop-position=12345 /var/lib/
mysql/mysql-bin.000006 | mysql -u root -p jueg
```

Opción B. Por fecha y Hora.

```
bash
mysqlbinlog --stop-datetime="2026-10-13
16:20:00" /var/lib/mysql/mysql-bin.000006 |
mysql -u root -p juego
```

Pregunta 10 Auditoria que no se pueda desactivar fácilmente
Consigna (codigo en phpmyadmin):
Intenta, como si fueras un atacante con acceso a la cuenta de la aplicacion (no admin), hacer DROP TRIGGER auditoria_monedas; o modificar Auditoria_Monedas para borrar tu rastro; luego, usando dba_admin, revisa que privilegios le harian falta al usuario de la aplicación para que esto fuera imposible desde el inicio

- Parte 1 intento como atacante

-- Intento de eliminar el trigger de auditoria
DROP TRIGGER IF EXIST auditoria_monedas;

-- Borrar el rastro en la tabla auditoria
DELETE FROM Auditoria_Monedas;

-- Vaciar la tabla auditorias
TRUNCATE TABLE Auditoria_Monedas;

-- Modificar la estructura para que deje de registrar
ALTER TABLE Auditoria_Monedas DISABLE Keys; -- o cualquier ALTER

- Parte 2: Como dba_admin

-- Ver que privilegios tiene el usuario
SHOW GRANTS FOR 'usuario_aplicacion'@'%'; -- cambia el nombre real del usuario

Luego revoka privilegios

Quitar Privilegios para quitar triggers
REVOKE TRIGGER ON *.* FROM 'usuario_aplicacion'@'%';

Quitar privilegios sobre la tabla auditoria
REVOKE DELETE, UPDATE, DROP, ALTER, TRUNCATE ON Auditoria_Monedas FROM 'usuario_aplicacion'@'%';

Quitar tambien privilegios globales peligrosos
REVOKE DROP, ALTER, CREATE, SUPER ON *.* FROM 'usuario_aplicacion'@'%';